

Atividade 3 - Módulo 3: Fine-Tuning via API

Disciplina: Processamento de Dados e Fine-Tuning de Modelos · Prof. Ahirton Lopes, Ph.D.

Missão Prática #03

Esta atividade fecha o Módulo 3 inteiro: vocês vão aplicar, ao próprio contexto de trabalho, o processo completo de fine-tuning via API comercial gerenciada, upload, hiperparâmetro validado, automação e versionamento, os instrumentos construídos ao longo dos cinco vídeos deste módulo.

Tempo e custo esperados

Nos jobs reais de fine-tuning supervisionado medidos ao longo desta disciplina, um piloto levou entre 13 minutos e cerca de 1 hora e 40 minutos pra treinar, dependendo do volume de dados. O custo ficou na casa de poucos reais por job, não dezenas: o projeto inteiro desta disciplina, do piloto ao modelo final escalado, chegou a R\$60,62 em jobs de fine-tuning e inferência somados, conferido direto no billing real do Google Cloud. Reservem tempo de espera na agenda, mas não esperem gastar muito, mesmo somando todos os jobs.

Opcional: quer rodar chamada real no próprio projeto Google Cloud, não só acompanhar os jobs desta disciplina? Criar projeto, vincular billing, ativar API, autenticar e criar bucket, passo a passo, em [gravacao-m3.1/demos/gcp-setup-companion.md](#). Nenhum passo desta atividade depende disso: os Passos 1-3 rodam local, e o Passo 4 tem tanto a opção de simular quanto a de analisar um resultado real já publicado, sem precisar de conta nenhuma.

Passo 1 - Converta seu dataset pro formato do provedor

Peguem o dataset da Missão Prática número dois, ou um novo, do próprio contexto de trabalho de vocês, e convertam pro formato de fine-tuning supervisionado do provedor que escolherem. Se for Gemini Enterprise Agent Platform, o mesmo formato desta disciplina, contents com role user e model, cada um com uma lista de parts contendo o texto. Se for outro provedor, adaptem a estrutura, mas mantenham a mesma disciplina: instrução e entrada compõem a entrada do modelo, saída vira a resposta esperada, sem editar o texto bruto.

Sem contexto de trabalho pronto, e não quer simular? [gravacao-m3.4/demos/dataset-real-alternativo-companion.md](#) tem um dataset real e público (databricks-dolly-15k, licença CC-BY-SA-3.0) já processado de ponta a ponta: mapeado pro schema canônico do Módulo 2.1, deduplicado, balanceado, convertido pro formato Vertex AI e treinado num job de fine-tuning real. O resultado real desse job está documentado na entrega desta missão, como segundo exemplo além do case Amplitude. Opcional, vale a partir desta missão até o fim da disciplina.

Passo 2 - Validem hiperparâmetro antes de qualquer chamada real

Implementem uma função que valide época, taxa de aprendizado, e qualquer outro hiperparâmetro relevante do provedor escolhido, ANTES de qualquer chamada de rede. Depois, se for criar job de verdade, comparem o hiperparâmetro pedido com o que o job realmente aplicou, consultando o job já criado. Esse segundo passo é o que teria pego o incidente real desta disciplina, hiperparâmetro inválido sendo silenciosamente substituído por um default do provedor.

Passo 3 - Implementem acompanhamento com backoff, testado sem rede

Em JavaScript, implementem uma função que consulte o status de um job repetidamente até ele chegar num estado terminal, com backoff exponencial entre consultas, não martelando a API a cada segundo. A função de consulta e a função de espera precisam ser injetáveis, permitindo pelo menos um teste automatizado que rode sem depender de rede nem de esperar tempo de verdade, usando sequência de estado falsa.

Passo 4 - Gerem uma ficha de versionamento real

A entrega final: uma ficha de versionamento completa, com hash SHA-256 do conteúdo do dataset de vocês (não do nome do arquivo), hiperparâmetro aplicado, modelo base, modelo ajustado e endpoint (reais, se um job de verdade foi criado; documentados como pendentes, se a decisão foi não criar job real), e estatística básica do dataset. Gerem também um model card em markdown, a partir dessa ficha.

Segundo exemplo real, pra quem usou o dataset alternativo do Passo 1: job real rodado contra 200 exemplos reais do databricks-dolly-15k, estado JOB_STATE_SUCCEEDED, endpoint publicado, 13min09s de duração. Este material passou por um painel de avaliação depois de publicado, que achou hiperparâmetro mal escolhido e dedup incompleto; ambos foram corrigidos e o job foi refeito, com resultado real melhor (comparação lado a lado no model card, mesmo dataset, hiperparâmetro diferente). Model card completo em [gravacao-m3.4/demos/model-card-dolly-extra-200.md](#).

Entrega

- Dataset convertido pro formato do provedor escolhido (Passo 1)
- Função de validação de hiperparâmetro, rodando antes de qualquer chamada de rede (Passo 2)
- Comparação pedido versus aplicado, se um job real foi criado (Passo 2)
- Acompanhamento com backoff exponencial, com teste automatizado sem rede (Passo 3)
- Ficha de versionamento completa, com hash real do dataset, e model card em markdown (Passo 4)
- Código funcional em JavaScript, com testes automatizados cobrindo os quatro passos

Dica de execução

Criar um job real de fine-tuning custa dinheiro e leva tempo, mesmo que pouco. Se o orçamento ou o tempo de vocês não permitir, documentem essa decisão com clareza no próprio código, simulem a chamada de criação com uma função falsa, e concentrem o esforço real nos Passos 1, 2 e 3, que não exigem gasto nenhum. O Passo 4 pode usar um job simulado, desde que a ficha gerada seja estruturalmente completa e o hash do dataset seja real, calculado sobre um arquivo de verdade, não inventado.

Alternativa ao Passo 4: entrega por análise, sem executar nada

Além de simular (dica acima), existe uma terceira opção pro Passo 4, pensada pra quem não tem acesso à Google Cloud e prefere não simular um resultado: analisar um resultado real que já existe, publicado nesta própria disciplina.

Este módulo já tem dois jobs de fine-tuning reais, rodados contra o mesmo dataset (hash SHA-256 idêntico, confirmado por reprodução independente), com um único hiperparâmetro

diferente entre eles: `learning_rate_multiplier` igual a um (v1) e igual a cinco (v2). O resultado observado muda de uma resposta alucinada (v1, com CEO e rota inventados) pra uma resposta correta e fiel ao contexto (v2). Os dois estão documentados de ponta a ponta em `gravacao-m3.4/demos/model-card-dolly-extra-200.md` e na seção 6 do companion (`dataset-real-alternativo-companion.md`).

Pra essa entrega, preencham a mesma ficha de versionamento do Passo 4 usando os dados reais do job v2 como objeto de estudo (hash, hiperparâmetro, modelo, endpoint, tudo real, só que de um job que vocês não rodaram), e respondam por escrito, com a mesma profundidade técnica que vocês dariam analisando o próprio job:

1. Qual foi a única variável que mudou entre v1 e v2, e por que ela é a explicação mais provável pro sintoma observado? Cite o raciocínio do Módulo 3.3 sobre multiplicador baixo andar devagar demais pra produzir ajuste perceptível num dataset pequeno.
2. Por que o hash de dataset idêntico entre as duas versões importa pra confiar nessa conclusão? O que enfraqueceria o argumento se os datasets fossem diferentes?
3. Se vocês fossem propor um terceiro job (v3) pra esse mesmo dataset, qual hiperparâmetro mexeriam a seguir, e que efeito específico esperariam ver mudar? Não vale responder "melhorar o modelo": a resposta precisa apontar um hiperparâmetro real do pipeline e um resultado observável esperado.

Essa entrega vale tanto quanto rodar, ou simular, o próprio job: o que o Passo 4 pede é raciocínio sobre a relação entre hiperparâmetro e resultado observado, e o par v1/v2 já publicado dá material real suficiente pra isso, sem depender de acesso a GCP nem de dataset de trabalho próprio.

Ahirton Lopes · Fine-Tuning Toolkit - UNIPDS: Processamento de Dados e Fine-Tuning de Modelos
Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager